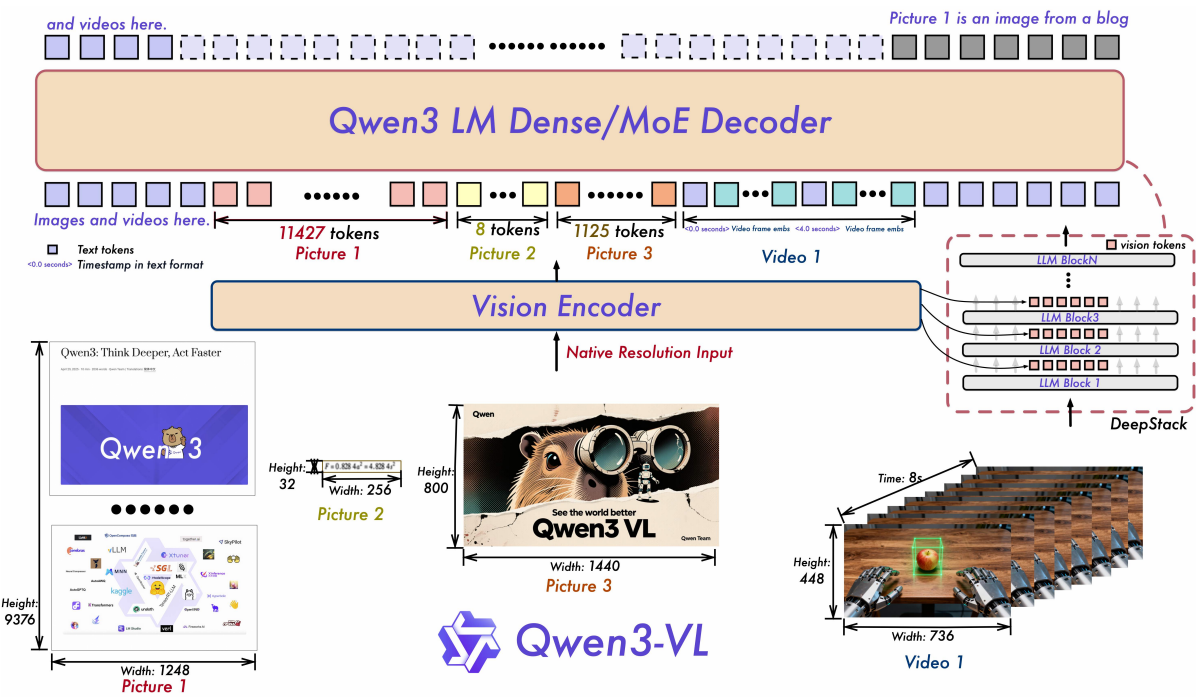


Qwen3-VL-2B Inference Optimization Report — IEEE AICAS 2026

Our optimization process follows an iterative loop: **algorithmic design** → **graph-level compilation & optimization** → **trace-driven operator/code optimization** → **repeat**.



Prefix KV Cache: Efficient KV Cache Management and Reuse

The token sequence structure is: `<pad>` + system prompt tokens + `<pad>` + image token placeholders + `<pad>` + user text tokens + `<pad>`

Key design principles:

1. Since image token placeholders precede text tokens, text-only prefix matching is not feasible in isolation.
2. All image placeholder tokens share the same token ID (`151655`). Image token segments cannot be split — they must be reused as whole segments.
3. The target hardware provides 98 GB of VRAM, offering abundant memory for caching.

Two Cache Data Structures

Radix Cache

A radix tree caches the KV of text prefixes. The fundamental unit of the text prefix cache is `CacheBlock`, with a default `block_size = 16`.

Each block stores:

- `token_ids`
- Per-layer `key_cache` and `value_cache` on GPU
- `rope_deltas`
- Block hash

The KV tensor shape per layer is: `[batch=1, num_kv_heads, block_size, head_dim]`. For Qwen3-VL 2B, this yields approximately: 28 layers $\times$ 2 (K/V) $\times$ 8 kv_heads $\times$ 16 tokens $\times$ 128 dim $\times$ bf16 $\approx$ 1792 KB per block.

```
root
├─ block0(token[0:16])
│   └─ block1(token[16:32])
│       └─ block2(token[32:48])
```

Image KV Cache

```
class ImageKVEntry:
    def __init__(self, img_hash, img_start, img_end,
                 key_cache, value_cache, rope_deltas=None):
        self.img_hash = img_hash
        self.img_start = img_start
        self.img_end = img_end
        self.img_len = img_end - img_start
        self.key_cache = key_cache
        self.value_cache = value_cache
        self.rope_deltas = rope_deltas

ImageKVCache = {
    img_hash_1: ImageKVEntry(...),
    img_hash_2: ImageKVEntry(...),
    ...
}
```

Optimization Details:

1. **Radix cache insertion optimization:** Instead of cloning K/V individually for each block, each layer, and each K/V separately, we first clone the entire newly-added KV region once per layer, then create block views via slicing. For a long prompt of 1000 tokens, this reduces approximately 62 blocks $\times$ 28 layers $\times$ 2 = 3,472 small clone operations down to

28 layers $\times$ 2 = 56 large clone operations, significantly reducing small kernel launch and Python scheduling overhead.

2. **KV cache reconstruction optimization:** On the first prefix hit, `reconstruct_kv_cache(path_nodes)` concatenates the blocks along the path into a `DynamicCache` along the sequence dimension:

- `layer_i_K = cat(block0.K_i, block1.K_i, ..., dim=seq)`
- `layer_i_V = cat(block0.V_i, block1.V_i, ..., dim=seq)`

A subsequent `_reconstruct_cache` is applied at the leaf node identified by `id(path_nodes[-1])`, so subsequent prefix hits avoid repeated concatenation.

3. **Dict-based image KV cache:** Text KV involves requests that may share prefixes of varying lengths (16, 32, 48, ... tokens), requiring longest-prefix matching — hence a Radix Tree is appropriate. Image KV only requires checking "has this image been seen before?", which is an exact-match lookup — so `dict[image_hash]` delivers $O(1)$ lookup without the overhead of a tree structure. Furthermore, image placeholder tokens all share the same ID (151655), so a token-sequence-based Radix Tree would incorrectly match different images. The true differentiator is `pixel_values`, making `image_hash(pixel_values)` the correct key.

Three-Tier Cache Reuse Design

1. **Image KV Cache:** Cache image-related KV using content hash for repeated image scenarios — skips vision/image prefill when the same image appears again.
2. **Text Prefix Matching (after image cache hit):** After an image cache hit, perform text prefix matching (same image, different text).
3. **Full Prompt KV Cache:** On a complete match, directly pass:
 - Per-layer K/V of the full prompt
 - `rope_deltas`
 - First generated token (`next_token`)

Model Compression and Pruning

Vision Encoder Layer Dropout

A body of research indicates that Vision Encoders in VLMs often exhibit redundancy (e.g., [arXiv:2506.08008](#), [arXiv:2507.03262](#)).

Qwen3-VL's vision encoder has 24 ViT blocks: `vision_depth = 24`, with `deepstack_visual_indexes = [5, 11, 17]`.

We retain only 7 layers — `[1, 5, 7, 11, 14, 17, 21]` — and observe that accuracy loss remains controllable (virtually no degradation).

Text Decoder MLP Channel Pruning

This approach targets reducing the computation and memory bandwidth of each decoder layer's FFN/MLP.

```
# Qwen3-VL text decoder per-layer MLP:
#   hidden_size = 2048
#   intermediate_size = 6144
#
# Standard structure:
#   gate = gate_proj(x)      # [*, 2048] -> [*, 6144]
#   up   = up_proj(x)       # [*, 2048] -> [*, 6144]
#   act  = silu(gate) * up   # [*, 6144]
#   out  = down_proj(act)   # [*, 6144] -> [*, 2048]
```

Channel pruning targets the intermediate dimension (6144 channels). We first run the model on a small calibration dataset, hook each layer's MLP, and compute per-channel importance scores.

The basic metric is: `abs(silu(gate_proj(x)) * up_proj(x)).mean()` — i.e., the mean activation magnitude of each intermediate channel on real inputs. A more refined metric incorporates the down_proj column norm:

```
importance(channel) ≈ activation_magnitude(channel) × ||down_proj[:,
channel]||
```

This accounts for the fact that a channel with high activation but very small down_proj weights contributes little to the final output. The resulting importance scores are saved to a file and applied as a runtime mask.

Input-Side Compression: Image Pixel Cap

```
max_pixels = 327680
image_processor.max_pixels = int(max_pixels)
```

Reducing the number of patches → shorter prompt length → lower TTFT.

Compilation and Graph-Level Optimization

All modules are compiled via `torch.compile`, followed by fine-grained graph replay for each module.

1. **Decode CUDA Graph:** Pre-captures fixed decode buckets to reduce Python scheduling and kernel launch overhead. We use 14 decode graph buckets. In validation, a random sample of 150 decode steps achieved 100% graph hit rate.
2. **Prefill CUDA Graph:** Captures prefill shapes into graphs to reduce TTFT. Uses shape-bucket-based pre-capture with up to 320 max entries. Observed `prefill_full` hit rate: 300/300.
3. **Segmented Prefill Graph:** Partitions text prefill into segmented graphs to reduce the difficulty of capturing and reusing large graphs. When the KV cache is partially hit, the vision encoder is skipped, and only the text-graph replay path is taken.

Operator-Level Optimizations

Prefill Stage

Operator	Implementation	Description
<code>prefill_rotary_triton</code>	Triton kernel, autotune (BLOCK_SIZE 256/512/1024)	Fuses Q and K rotary position embedding into a single Triton kernel launch, replacing two separate <code>apply_rotary_pos_emb</code> calls
<code>prefill_rmsnorm_triton</code>	Triton kernel, dynamic block_size (power of 2, max 4096)	Row-wise RMSNorm: <code>rsqrt(mean(x²)+eps) * x * weight</code> . Replaces <code>Qwen3VLTextRMSNorm.forward</code> across 28 layers (input_layernorm / post_attention_layernorm / q_norm / k_norm)
<code>prefill_qk_rmsnorm_rotary_cuda</code>	CUDA kernel, hardcoded head_dim=128, block_reduce_128	Triple fusion: Q RMSNorm + K RMSNorm + QK Rotary in a single CUDA kernel. Grid = seq × (q_heads + k_heads) blocks, one (token, head) pair per block
<code>flashAttention_tilelang</code>	TileLang compiled to CUDA (.so), pre-compiled per seq_len	Pre-compiled FlashAttention kernels for specific text prefill lengths (640, 704, 768, 832, 1024, 1088), routed via <code>dispatch_policy.json</code> . Replaces <code>F.scaled_dot_product_attention</code> , B=1, Hq=16, Hkv=8, D=128, causal=True

Decode Stage

Operator	Implementation	Description
FlashDecode Attention	CUDA PTX kernel, C++ runner, single/multi-query modes	High-performance decode attention. Single-query uses <code>run_decode_into</code> ; multi-query causal uses <code>run_decode_multi_into</code> ; multi-query masked uses a root-split strategy (root → single query, rest → masked multi-kernel). KV length range: [595, 2067]

FlashDecode FFN	C++/CUDA packed-weight GEMV, three backends (v3/cublaslt/legacy)	Fuses gate_proj + up_proj + SiLU + down_proj into a single kernel. Uses <code>pack_w13</code> (gate+up interleaved packing) and <code>pack_w2</code> (down transposed packing) for optimized memory access. Wrapped with <code>torch.compiler.disable()</code> for CUDA Graph compatibility
<code>decode_qk_rotary</code> CUDA ext	CUDA extension (PTX MMA), supports RoPE cache precomputation	Replaces the entire Q/K projection + RoPE pipeline in the decode stage. CUDA Graph-compatible linear mode, accelerated via <code>decode_linear_cublaslt</code> or <code>decode_linear_triton</code> GEMV
Decode ROPE Cache	CUDA kernel (<code>rope_cache_lookup</code>), precomputed cos/sin	Precomputes cos/sin caches in GPU memory (<code>max_len=4096</code>). During decode, performs direct table lookup to avoid repeated RoPE frequency computation
<code>decode_add_rmsnorm</code> Triton	Triton kernel, autotune; $n \leq 4096$ single kernel, $n > 4096$ three-stage tiled	Dual fusion: residual + attn_out addition and RMSNorm normalization in a single kernel
<code>lm_head_argmax</code> Triton	Triton kernel, block-wise max + final reduction	Fuses lm_head linear projection + argmax into a single kernel, directly outputting token IDs and avoiding materialization of full vocabulary logits ($V \approx 152,064$). Uses vocabulary chunking: per-chunk max + position, final reduction for global argmax
Decode Linear cuBLASLt	cuBLASLt MatMul + algorithm cache, or hand-written PTX MMA	GEMV acceleration for QKV linear projections. Two strategies: (1) cuBLASLt with algorithm caching, (2) hand-written MMA PTX assembly tensor-core GEMV. Uses the MMA path when $M \leq 16$

Vision Encoder Stage

Operator	Implementation	Description
<code>vision_attn_triton</code>	Triton kernel, online softmax, TM=TN=32, H=16, D=64	Non-causal multi-head attention for Qwen3-VL-2B ViT. Uses online softmax to avoid materializing the large attention matrix. Replaces PyTorch SDPA
<code>vision_mlp_fused_triton</code>	Triton kernel, GELU-tanh + down_proj fusion, BLOCK_H=32, BLOCK_I=128	Fuses GELU-tanh activation + linear_fc2 down-projection in ViT MLP into a single Triton kernel. One program per token row

KV Cache Operations

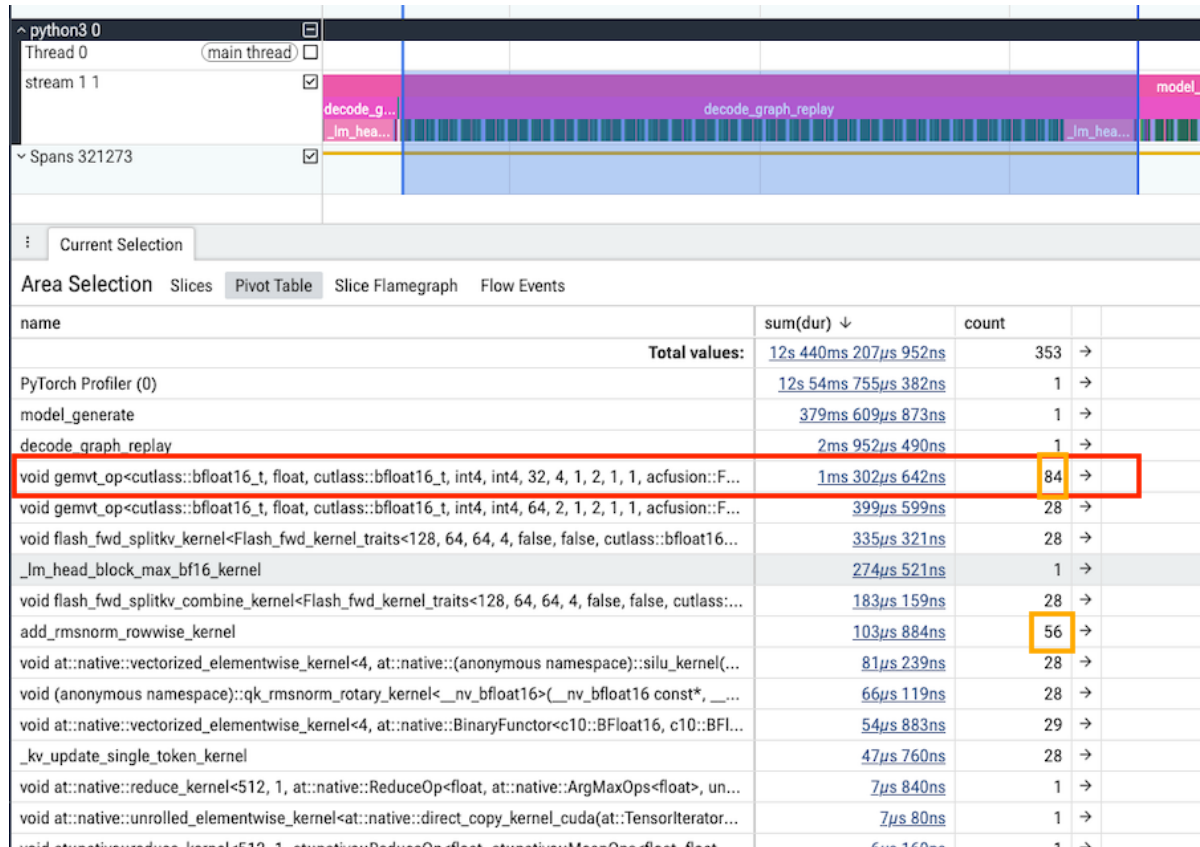
Operator	Implementation	Description
<code>static_cache_update</code> Triton	Triton kernel, single-token / multi-token modes	Replaces <code>StaticLayer.update()</code> , writing K/V directly to the static cache at the specified <code>cache_position</code> via Triton, avoiding PyTorch scatter/index overhead. Multi-token mode supports speculative decoding (max 16 tokens)
<code>static_kv_copy</code> CUDA	CUDA kernel, uint4 vectorized (16-byte) or scalar	Copies full-prompt KV from temporary tensors to static cache. Uses uint4 vectorized load/store when <code>head_dim</code> is aligned (divisible by 8) for significant acceleration

General / Fusion Optimizations

Operator	Implementation	Description
<code>embedding_gather</code> Triton	Triton kernel, BLOCK_H=256	Replaces <code>nn.Embedding.forward</code> , directly gathering token embeddings from the weight matrix via Triton to avoid PyTorch embedding's grouped launch pattern

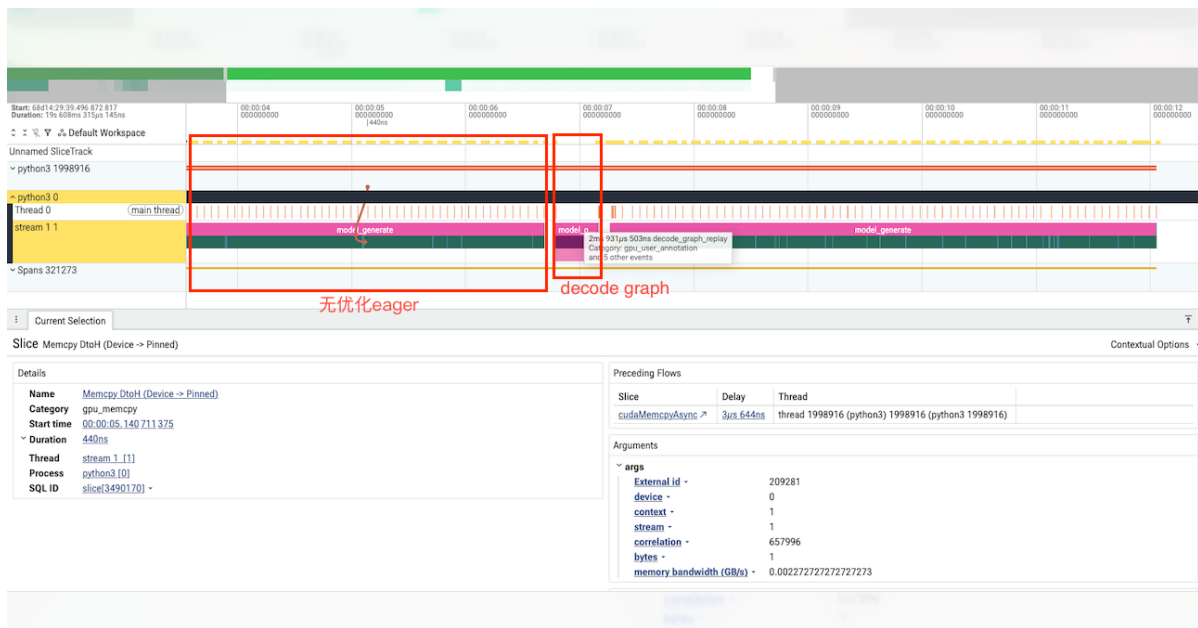
Trace-Guided Operator and Code Optimization

1. Identifying and Replacing Hotspot Operators



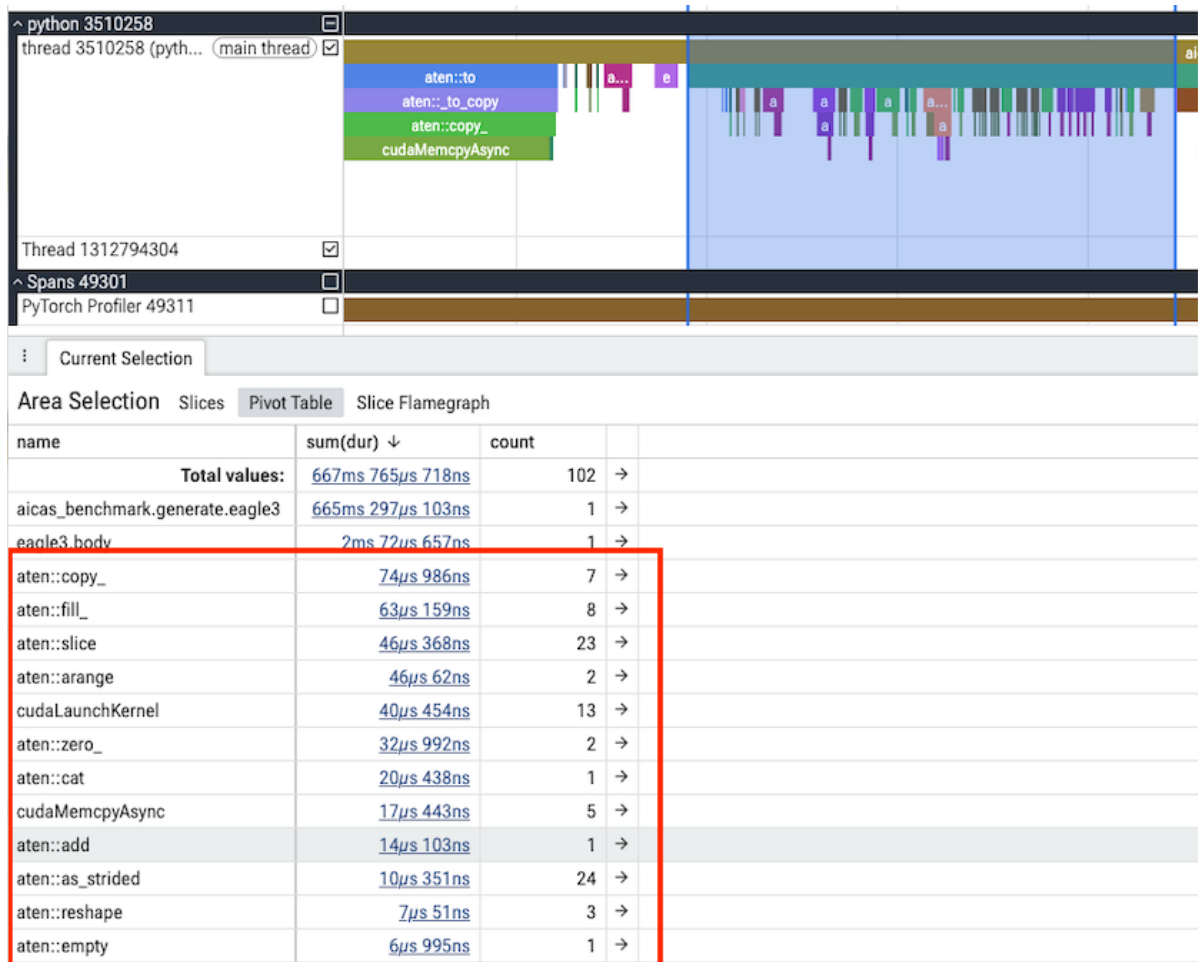
- GEMV is the dominant hotspot operator. Optimizing GEMV — through shape-specific tuning, quantization, and maximizing tensor-core utilization — yields significant gains.
- By counting per-decode-step operator invocations, we verify that each of the 28 layers should produce 28 calls. Redundant call counts expose opportunities for operator fusion and code optimization.

2. Reducing Device-to-Host (D2H) Transfers



- While individual, small D2H transfers may not incur large overhead, the forced `cudaMemcpySync` they introduce prevents graph capture. We systematically reworked data flows to keep all data paths on the GPU device side, minimizing synchronization overhead.

3. Operator Fusion: Eliminating Small-Kernel Launch Overhead



- Numerous small kernels generate excessive kernel launch overhead and inter-kernel gaps. By analyzing trace graphs to locate these patterns in the code, we applied targeted fusion to consolidate them into larger, more efficient kernels.